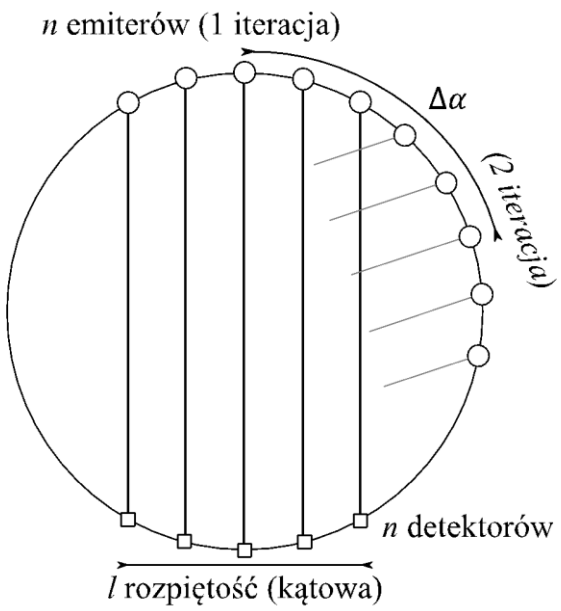
14.04.2026, Poznań

Sprawozdanie z symulatora tomografu komputerowego

1. Skład grupy:
    a. Kamil Suwiczak 160398
    b. Patryk Migaj 160868

2. Zastosowany model tomografu
   Model równoległy



3. Zastosowane technologie
    a. Python
        i. Streamlit
        ii. NumPy
        iii. Pydicom
        iv. Numba
        v. Datetime
        vi. PIL
        vii. Pandas
        viii. Altair

4. Główne funkcje programu
    a. pozyskiwanie odczytów dla poszczególnych detektorów

```
def radon_transform(image_array, n_scans, n_detectors, fan_angle_deg, progress_bar=None):
    height, width = image_array.shape
    center_x, center_y = width // 2, height // 2
    radius = math.sqrt(center_x ** 2 + center_y ** 2)
    fan_angle = math.radians(fan_angle_deg)

    sinogram = np.zeros((n_scans, n_detectors))
```

```
    for scan in range(n_scans):
        alpha = scan * (math.pi / n_scans)
        emitters, detectors = _get_positions_numba(radius, alpha, fan_angle, n_detectors,
center_x, center_y)

        for i in range(n_detectors):
            x0, y0 = emitters[i]
            x1, y1 = detectors[i]
            sinogram[scan, i] = _ray_mean_numba(image_array, x0, y0, x1, y1, width,
height)

        if progress_bar:
            progress_bar.progress((scan + 1) / (2 * n_scans), text="Generowanie
sinogramu...")

    return sinogram
```

b. filtrowanie sinogramu, zastosowany rozmiar maski

```
def generate_filter(size):
    kernel = np.zeros(size)
    center = size // 2
    for i in range(size):
        k = i - center
        if k == 0:
            kernel[i] = 1.0
        elif k % 2 == 0:
            kernel[i] = 0.0
        else:
            kernel[i] = -4.0 / (np.pi ** 2 * k ** 2)
    return kernel

def apply_filter(sinogram):
    filtered = np.zeros_like(sinogram)
    num_detectors = sinogram.shape[1]
    kernel = generate_filter(num_detectors * 2 + 1)

    for i in range(sinogram.shape[0]):
        row = sinogram[i, :]
        conv_full = np.convolve(row, kernel, mode="full")
        center_start = (len(conv_full) - len(row)) // 2
        filtered[i, :] = conv_full[center_start: center_start + len(row)]

    return filtered
```

c. ustalanie jasności poszczególnych punktów obrazu wynikowego oraz jego

przetwarzanie końcowe

```
def normalize_robust(image):
    img_flat = image.flatten()

    p_low = np.percentile(img_flat, 2)
```

```
    p_high = np.percentile(img_flat, 98)

    clipped = np.clip(image, p_low, p_high)

    if p_high == p_low:
        return np.zeros_like(clipped)

    return (clipped - p_low) / (p_high - p_low)

def _backproject_ray_numba(reconstructed, weight_matrix, value, x0, y0, x1, y1, width,
height):
    points = _bresenham_numba(x0, y0, x1, y1, width, height)
    for i in range(points.shape[0]):
        x = points[i, 0]
        y = points[i, 1]
        reconstructed[y, x] += value
        weight_matrix[y, x] += 1


def simulate_tomograph(
        image_array,
        n_scans,
        n_detectors,
        fan_angle_deg,
        use_filter=False,
        progress_bar=None,
        collect_rmse_per_iteration=False,
):
    height, width = image_array.shape
    center_x, center_y = width // 2, height // 2
    radius = math.sqrt(center_x ** 2 + center_y ** 2)
    fan_angle = math.radians(fan_angle_deg)

    sinogram = radon_transform(image_array, n_scans, n_detectors, fan_angle_deg,
progress_bar)

    display_sinogram = sinogram.copy()

    if use_filter:
        sinogram = apply_filter(sinogram)

    sinogram = normalize_array(sinogram)

    reconstructed = np.zeros((height, width))
    weight_matrix = np.zeros((height, width))
    rmse_per_iteration = [] if collect_rmse_per_iteration else None

    for scan in range(n_scans):
        alpha = scan * (math.pi / n_scans)
        emitters, detectors = _get_positions_numba(radius, alpha, fan_angle, n_detectors,
center_x, center_y)

        for i in range(n_detectors):
            x0, y0 = emitters[i]
```

```
            x1, y1 = detectors[i]
            value = sinogram[scan, i]
            _backproject_ray_numba(reconstructed, weight_matrix, value, x0, y0, x1, y1,
width, height)

        if collect_rmse_per_iteration:
            current = np.zeros_like(reconstructed)
            np.divide(reconstructed, weight_matrix, out=current, where=weight_matrix != 0)

            current = normalize_robust(current)
            rmse_per_iteration.append(calculate_rmse(image_array, current))

        if progress_bar:
            progress_bar.progress(0.5 + (scan + 1) / (2 * n_scans), text="Rekonstrukcja
obrazu...")

    np.divide(reconstructed, weight_matrix, out=reconstructed, where=weight_matrix != 0)

    reconstructed = normalize_robust(reconstructed)

    if collect_rmse_per_iteration:
        return display_sinogram, reconstructed, rmse_per_iteration
    return display_sinogram, reconstructed
```

d. wyznaczanie wartości miary RMSE na podstawie obrazu źródłowego oraz wynikowego

```
def calculate_rmse(image1, image2):
    return np.sqrt(np.mean((image1 - image2) ** 2))
```

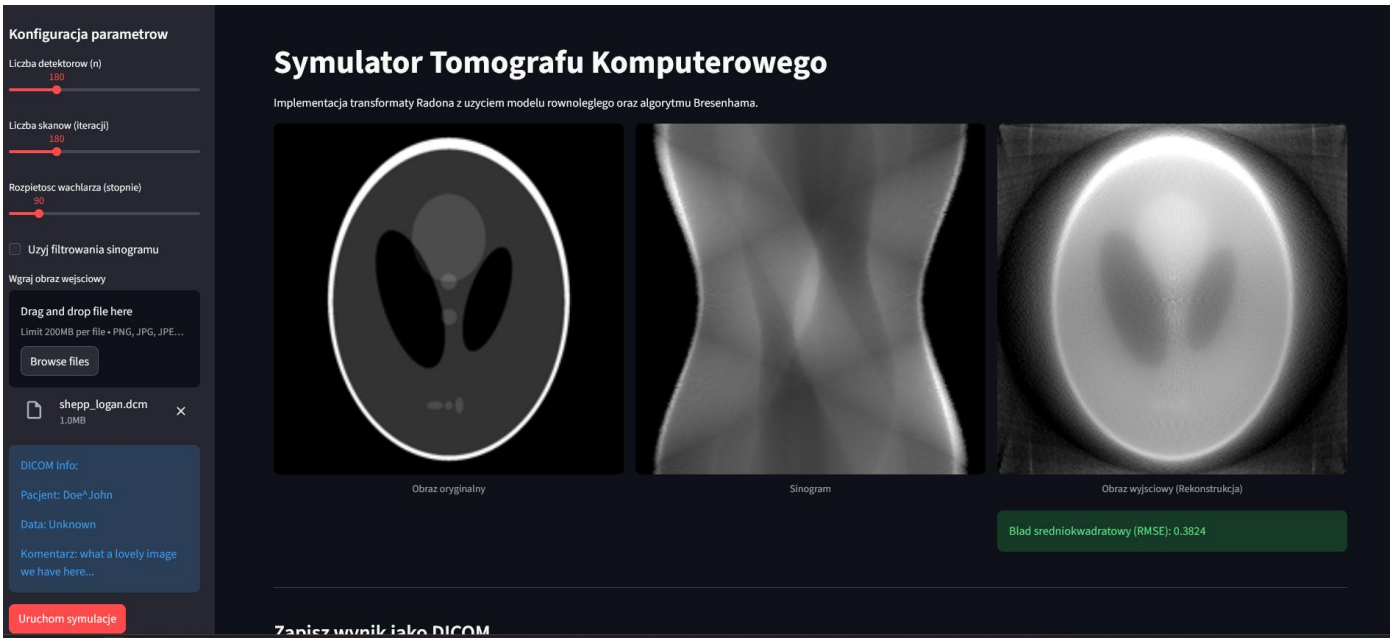
e. odczyt i zapis plików DICOM

```
def save_dicom(image_array, patient_name, comment, date_str):
    image_16bit = (image_array * 65535).astype(np.uint16)

    file_meta = FileMetaDataset()
    file_meta.MediaStorageSOPClassUID = '1.2.840.10008.5.1.4.1.1.2'
    file_meta.MediaStorageSOPInstanceUID = generate_uid()
    file_meta.ImplementationClassUID = generate_uid()
    file_meta.TransferSyntaxUID = ExplicitVRLittleEndian

    ds = FileDataset(None, {}, file_meta=file_meta, preamble=b"\0" * 128)

    ds.PatientName = patient_name
    ds.PatientID = "123456"
    ds.ContentDate = datetime.datetime.now().strftime('%Y%m%d')
    ds.ContentTime = datetime.datetime.now().strftime('%H%M%S.%f')[:10]
    ds.StudyDate = date_str.replace("-", "")
    ds.Modality = "OT"
    ds.ImageComments = comment
```

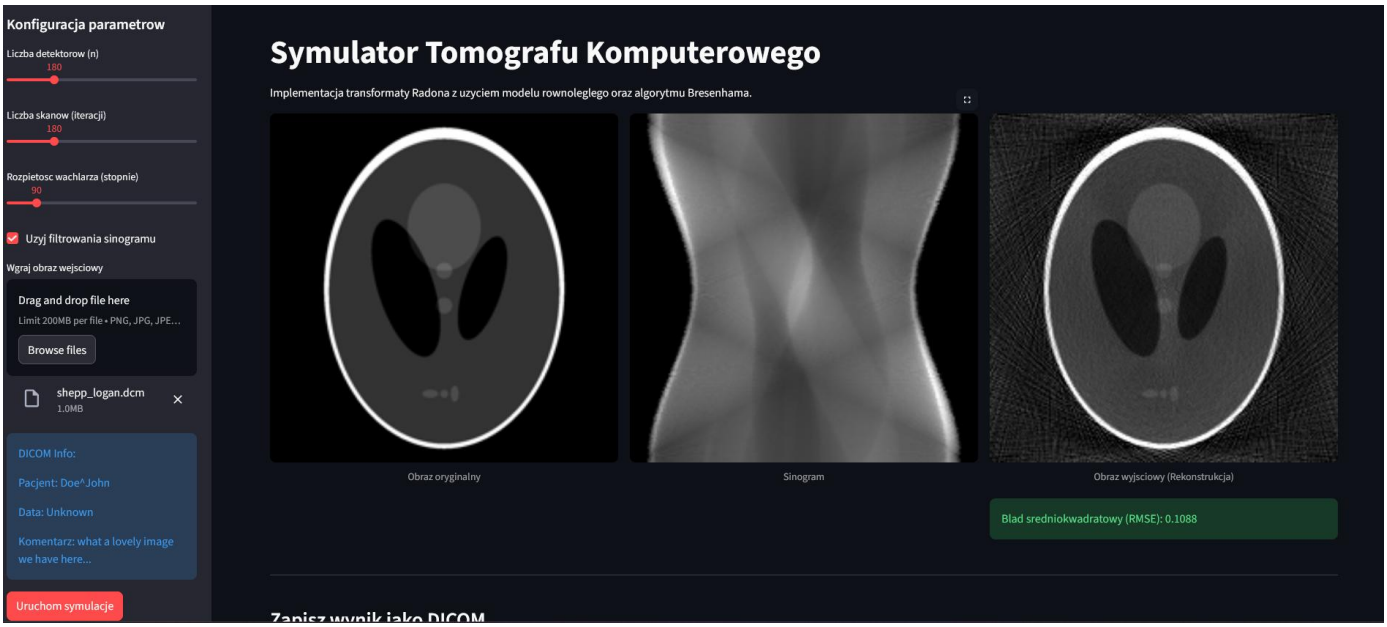
```
    ds.StudyInstanceUID = generate_uid()
    ds.SeriesInstanceUID = generate_uid()
    ds.SOPInstanceUID = file_meta.MediaStorageSOPInstanceUID
    ds.SOPClassUID = file_meta.MediaStorageSOPClassUID
    ds.InstanceNumber = "1"

    ds.Rows, ds.Columns = image_16bit.shape
    ds.SamplesPerPixel = 1
    ds.PhotometricInterpretation = "MONOCHROME2"
    ds.PixelRepresentation = 0
    ds.HighBit = 15
    ds.BitsStored = 16
    ds.BitsAllocated = 16

    ds.PixelData = image_16bit.tobytes()

    dicom_io = io.BytesIO()
    pydicom.dcmwrite(dicom_io, ds, write_like_original=False)
    dicom_io.seek(0)

    return dicom_io
```

```
def load_dicom(file_path):
    ds = pydicom.dcmread(file_path)

    patient_name = str(ds.PatientName) if 'PatientName' in ds else "Unknown"
    study_date = ds.StudyDate if 'StudyDate' in ds else "Unknown"
    comment = ds.ImageComments if 'ImageComments' in ds else ""

    image_array = ds.pixel_array.astype(np.float64)

    if ds.BitsAllocated == 16:
        image_array = image_array / 65535.0
    else:
        max_val = np.max(image_array)
        if max_val > 0:
            image_array = image_array / max_val

    return image_array, patient_name, comment, study_date
```

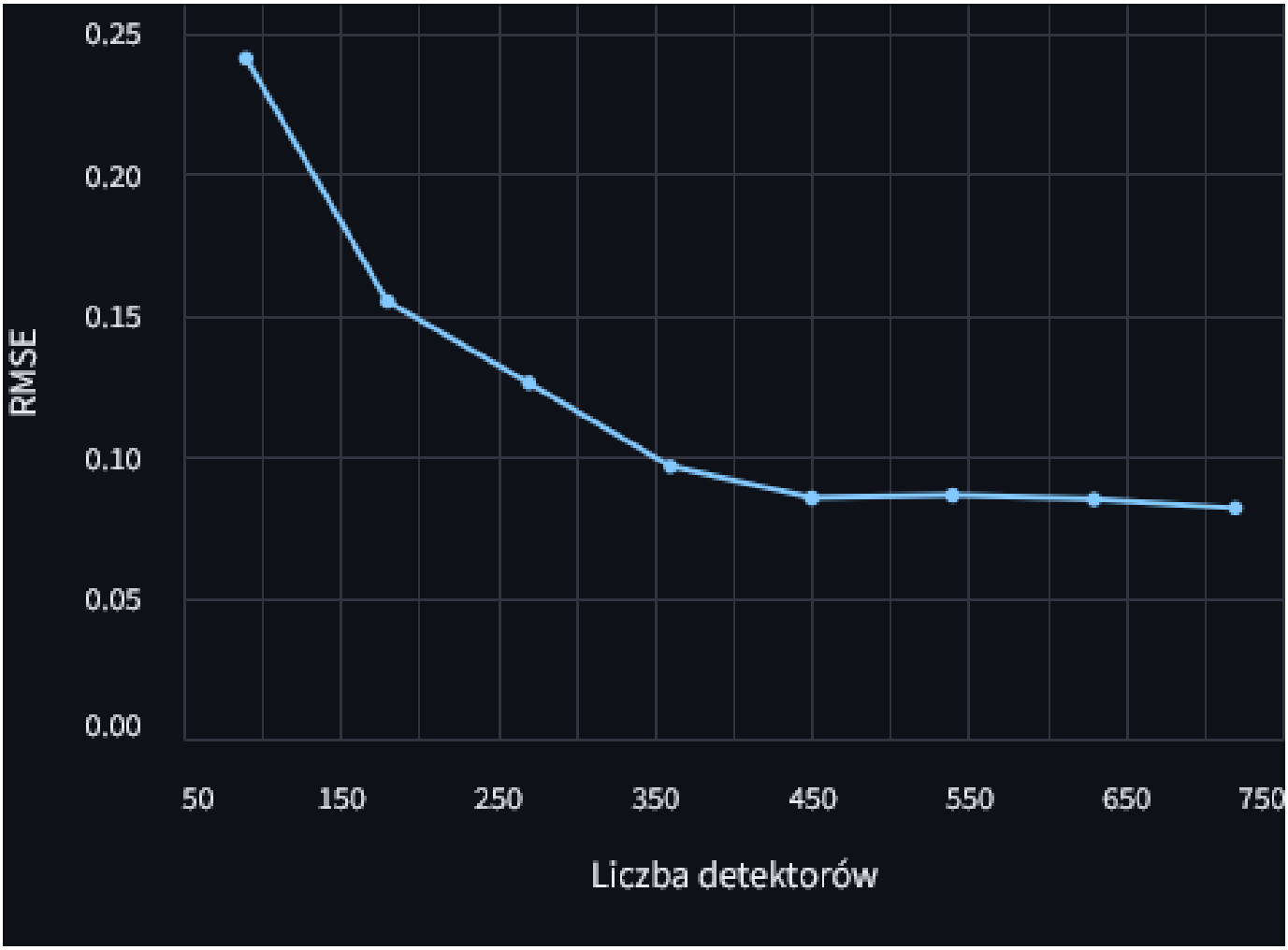
5. Przykład działania programu dla obrazu shepp_logan.jpg
    a. Symulacja bez filtrowania



    b. Symulacja z użyciem filtrowania

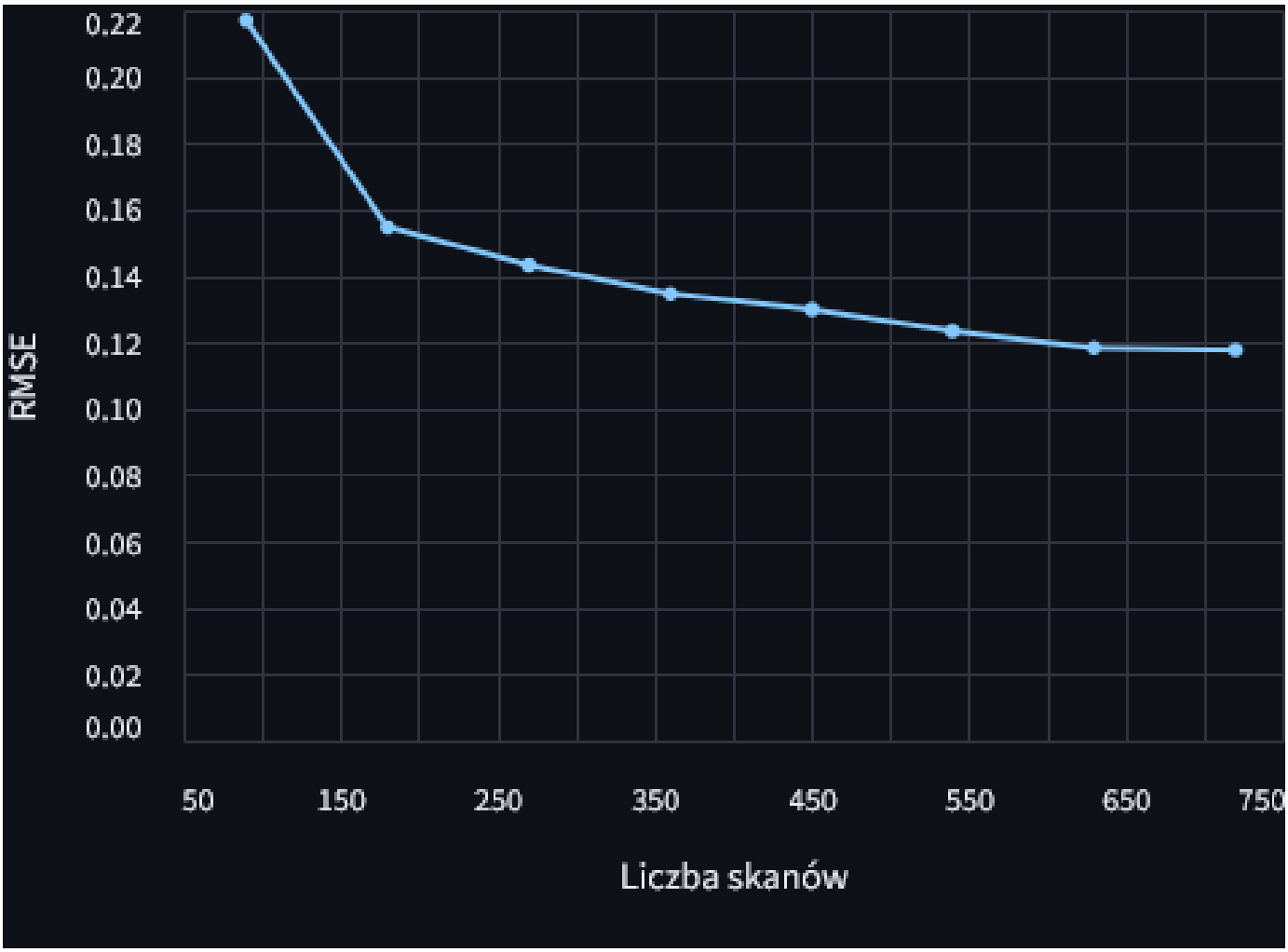


6. Wyniki eksperymentu sprawdzającego wpływ poszczególnych parametrów na podstawie zdjęcia shepp_logan.jpg bez włączonego filtrowania
    a. Zmiana liczby detektorów od 90 do 720 z krokiem 90



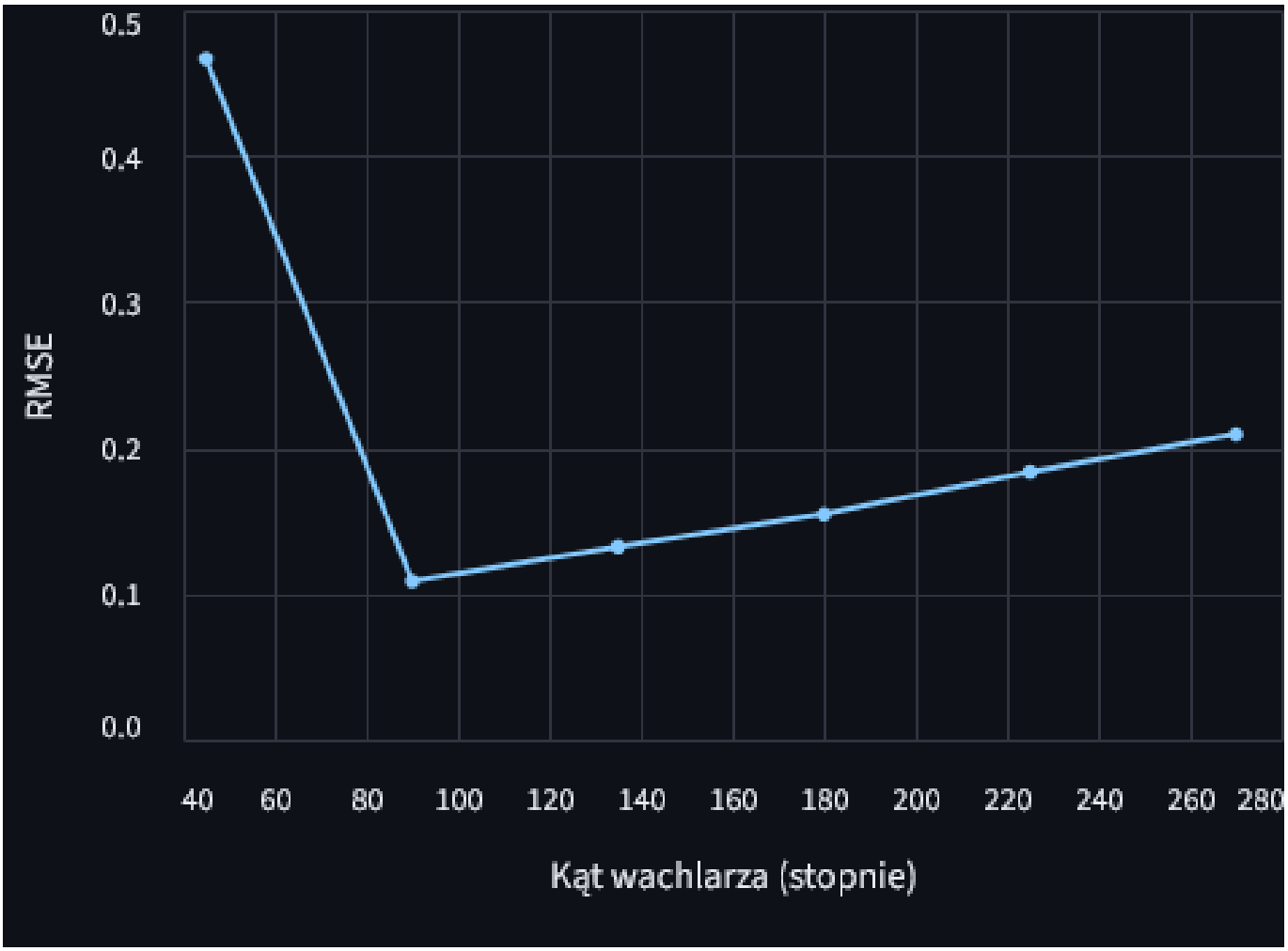
Największą poprawę widać podczas zmiany liczby detektorów z 90 na 180, następnie następuje powolny spadek tempa poprawy jakości na przedziale [180,360]. W przedziale [360,450] pogłębia się wypłaszczanie wykresu. Powyżej 450 detektorów wykres upodabnia się do stałej, co oznacza, że powyżej tej liczby detektorów nie ma już poprawy jakości

b. Zmiana liczby detektorów od 90 do 720 z krokiem 90



Największą poprawę jakości widać w przedziale [90,180], gdzie następuje nagły spadek wartości RMSE. Następnie wykres zamienia się w funkcję liniową ze znacznie mniejszym tempem poprawy jakości obrazu.

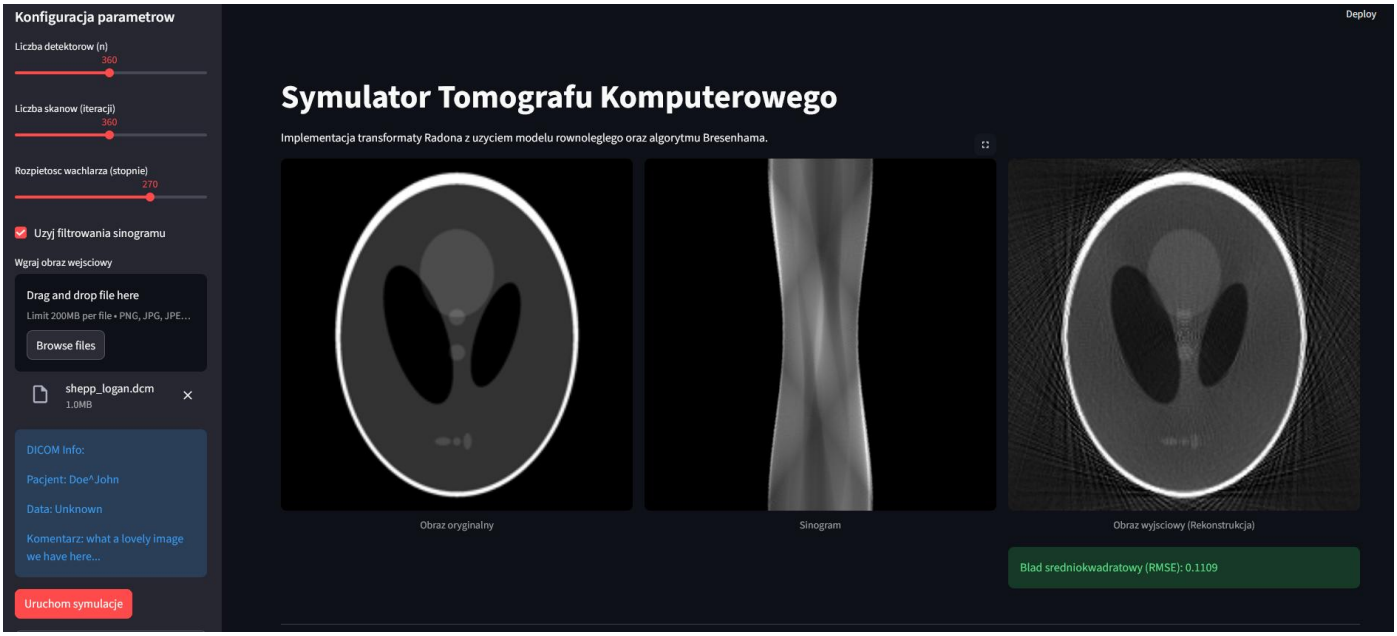
c. Zmiana liczby detektorów od 90 do 720 z krokiem 90



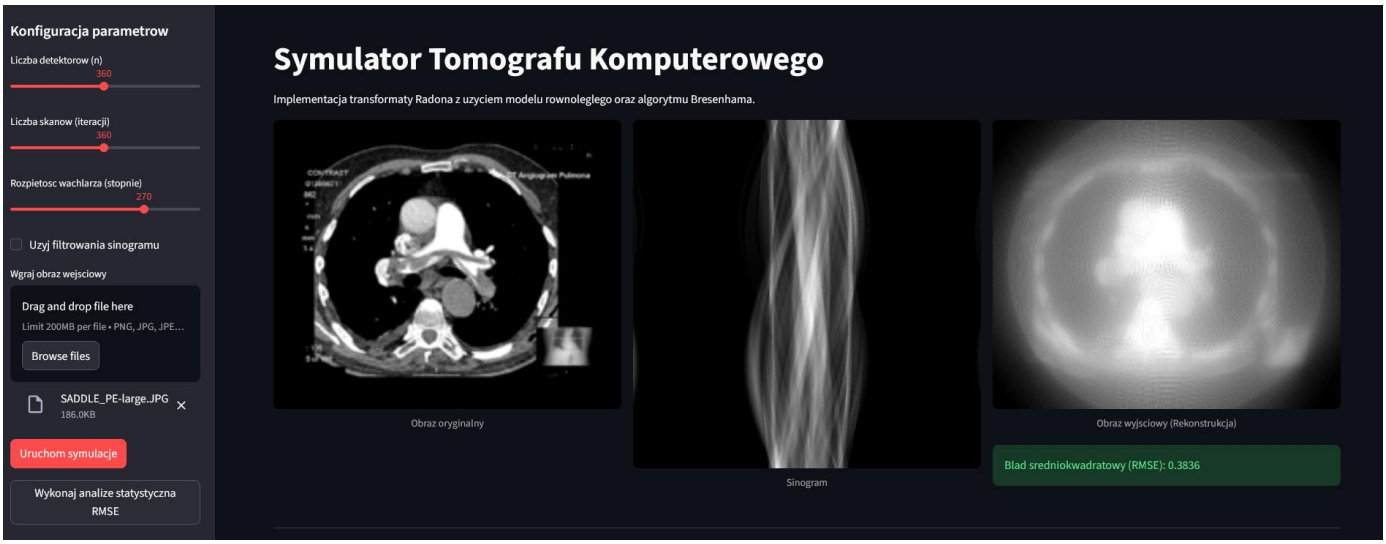
W przedziale [45°, 90°] następuje znaczne poprawienie jakości obrazu (wręcz wykładnicze). Powyżej wartości 90° następuje liniowe pogorszenie wartości RMSE, a co za tym idzie jakości obrazu.

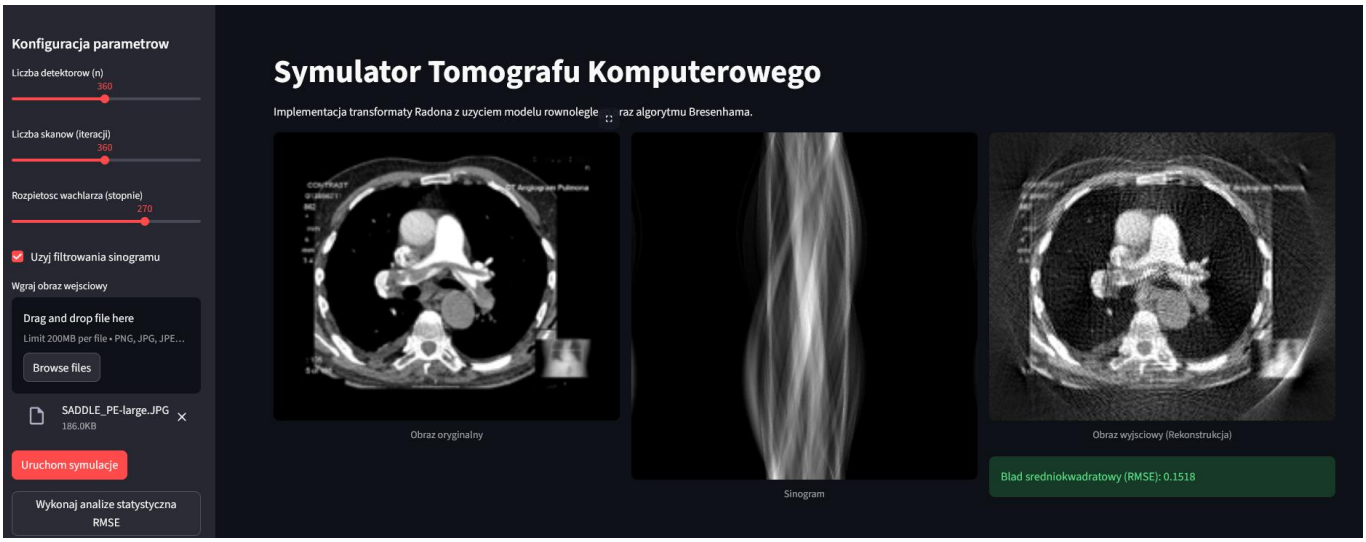
d. Porównanie symulacji dwóch obrazów dla liczby detektorów = 360, liczby skanów = 360, rozpiętość wachlarza = 270 stopni

Obraz 1 : shepp_logan.JPG



Obraz 2: SADDLE_PE-large.JPG





W przypadku obrazu pierwszego błąd średniokwadratowy podczas symulacji bez filtrowania wynosił 0,4941, po zastosowaniu filtrowania wynik ten poprawił się do 0,1109. W przypadku obrazu drugiego błąd średniokwadratowy bez filtrowania wynosił 0,3836, a po zastosowaniu filtra zmniejszył się do 0,1518.

Ponadto jest też znacząca różnica w wyglądzie. Przy obu obrazach brak filtra skutkował znacznie jaśniejszym obrazem wynikowym niż obraz wejściowy i szczegóły były mało widoczne. Po zastosowaniu filtra obydwa obrazy stały się ciemniejsze, znacznie bardziej czytelne i z większą ilością szczegółów.